

Documentación Proyecto de Sistemas Operativos

Integrantes del equipo

Gustavo Alonso Anaya Moreno

María Itzel Hernández Marín

Jaime Jaír Raya Castellón

Fernanda Montserrat Fabian Reyes

Estructura del proyecto

El proyecto es un simulador de sistema operativo. El usuario indica un archivo de programa (por ejemplo, a.asm), y el sistema lo carga, valida cada instrucción y la ejecuta paso a paso, mostrando en pantalla el contador de programa (PC), el registro de instrucción (IR), el valor de los registros AX, BX, CX y DX, y el estado de cada instrucción.

El lenguaje que interpreta admite siete instrucciones: MOV, ADD, SUB, MUL y DIV, que reciben dos parámetros (INSTR REG, VALOR), e INC y DEC, que reciben uno (INSTR REG). El valor puede ser un registro o un número entero. El formato debe respetarse exactamente: un solo espacio entre la instrucción y el registro, y ningún espacio antes ni después de la coma. El programa termina al llegar al final del archivo, sin necesidad de una instrucción de fin.

Estructura de datos

Un programa cargado se representa como una lista ligada de renglones. La estructura Archivo guarda el primer y el último renglón, además del número total de renglones. Cada Renglon contiene el texto original de la línea, el código de error detectado al cargarla y su propia lista ligada de tokens. Cada Token guarda una palabra de la instrucción y el tipo que le asignó el análisis léxico (instrucción, registro, registro con coma o número).

La estructura Ejecucion representa al programa en ejecución. Contiene el nombre del proceso, el programa cargado, el contador de programa (PC), un apuntador al renglón que se está ejecutando (IR), el estado (en ejecución o en espera) y un contador que controla la velocidad de ejecución. Los registros AX, BX, CX y DX se guardan en la estructura Registros, que existe como variable global (reg).

Flujo General

1. Inicio. main inicializa ncurses, crea las tres ventanas con crearVentana, dibuja el encabezado de la tabla con impEncabezado y crea los registros con crearRegistro. Después entra en un ciclo que se repite cada 16 milisegundos.

2. Lectura de comandos. En cada vuelta del ciclo, kbhit revisa si se presionó una tecla, sin detener el programa. Los caracteres se van mostrando con leerComando y, al presionar Enter, detectarComando identifica el comando escrito. Si es salir, el ciclo termina. Si no es válido, se muestra el error correspondiente.
3. Carga del programa. Con el comando ejecutar, se libera el programa anterior con liberarEjecucion y se prepara uno nuevo con crearArchivo y crearEjecucion, que también reinicia los registros en cero. Después se realizan tres pasos:
 - leerArchivo lee el archivo línea por línea y crea un renglón por cada una.
 - tokenizar procesa cada renglón. Primero detecta las líneas vacías y revisa el formato de espacios y coma con verifEspacios. Después separa el renglón en tokens y clasifica cada uno con agregarTipoDato, que recorre en cadena esInstruccion, esRegistro y esNumero.
 - verifSintaxis revisa que cada renglón empiece con una instrucción válida y tenga los parámetros correctos, usando parserDosParametros o parserUnParametro según la instrucción.
4. Cuando un renglón tiene un error, el código queda guardado en el propio renglón y el análisis continúa con los siguientes. La carga solo se cancela si el archivo no existe o está vacío.
5. Ejecución. Una vez cargado el programa, el IR apunta al primer renglón y el estado cambia a ejecución. Cada 125 vueltas del ciclo (aproximadamente dos segundos), ejecutarPrograma incrementa el PC y ejecuta el renglón actual llamando a la función de la instrucción (mov, add, sub, mul, divR, inc o dec). Estas funciones obtienen el registro destino con obtenerRegistro y el valor del segundo parámetro con obtenerValor. Si el renglón trae un error de la carga, no se ejecuta y solo se reporta.
6. Visualización. Después de cada instrucción, impInstruccVentana agrega una fila a la tabla con el estado de los registros y el resultado, y mensajeEstatus proporciona el texto de la columna Status. Si hubo un error, detectarError muestra el mensaje completo en la ventana de errores. Luego el IR avanza al siguiente renglón.
7. Fin. Cuando el IR llega al final de la lista, el estado cambia a espera y el sistema queda listo para un nuevo comando.

estructuras.h

Estructuras

Token (*Línea 4*)

Representa una palabra de una instrucción, ya sea la instrucción, un registro o un número. Guarda el texto de la palabra (textoToken), su tipo según el análisis léxico (tipoParam) y un apuntador al siguiente token del renglón (sig), de modo que los tokens de cada línea forman una lista ligada. Cuando el token es un registro seguido de coma, la coma se elimina del texto durante el análisis léxico. Se crea con crearToken dentro de tokenizarRenglon y se libera con liberarRenglon.

Renglon (*Línea 10*)

Representa una línea del archivo de programa. Guarda el texto original de la línea sin el salto de línea (texto), que es el que se muestra en la columna IR; el código de error detectado al cargarla (error), que vale BIEN si la línea es válida; el siguiente renglón (sig); y el primer y el último token de la línea (primerToken y ultimoToken). Se crea con crearRenglon durante la lectura del archivo y se libera con liberarRenglon, que también libera sus tokens.

Archivo (*Línea 18*)

Representa el programa completo cargado en memoria como una lista ligada de renglones. Guarda el primer renglón (inicio), el último (final), que permite agregar renglones sin recorrer la lista, y el número total de renglones (tamanio). Se crea con crearArchivo cada vez que se carga un programa y se libera con liberarArchivo.

Registros (*Línea 24*)

Guarda el valor de los cuatro registros del procesador simulado: EAX, EBX, ECX y EDX, que corresponden a AX, BX, CX y DX. Existe una sola instancia global, reg, compartida por todo el programa. Se crea con crearRegistro al iniciar, sus valores se reinician en cero cada vez que se carga un programa y se libera al terminar main. Las instrucciones acceden a cada registro mediante obtenerRegistro.

Ejecución (*Línea 31*)

Representa el programa en ejecución. Guarda el nombre del proceso, que es el nombre del archivo sin la ruta (nombre); el programa cargado (programa); el contador de programa (PC), que cuenta las instrucciones ejecutadas, incluidas las que tuvieron error; el registro de instrucción (IR), que apunta al renglón que se va a ejecutar y vale NULL al terminar el programa; el estado de la ejecución (estado); Id y un contador (espera) que hace que se ejecute una instrucción cada 125 vueltas del ciclo principal, aproximadamente cada dos segundos. Se crea con crearEjecucion y se libera con liberarEjecucion, que también libera el programa y el nombre.

Enumeraciones

tipoParametro (*Línea 41*)

Identifica el tipo de cada token según el análisis léxico. Sus valores son INSTR (1) para una instrucción válida, REG_COM (2) para un registro seguido de coma, que es el primer parámetro de las instrucciones de dos parámetros, REG (3) para un registro sin coma y NUM (4) para un número entero. Lo asignan las funciones del análisis léxico y lo consultan el análisis sintáctico y obtenerValor.

comandoTerminal. *(Línea 46)*

Identifica el comando que escribe el usuario: SALIR (10) termina el programa y EJECUTAR_ARCHIVO (11) carga y ejecuta un archivo. Lo regresa detectarComando y lo usa main para decidir qué hacer.

estadoSistema *(Línea 51)*

Indica si hay un programa ejecutándose. EJECUCION (50) significa que hay un programa ejecutando sus instrucciones, y ESPERA (51) que el sistema está esperando un comando. Una ejecución inicia en espera, pasa a ejecución al cargarse el programa y regresa a espera al llegar al final del archivo.

errorAdvertencia *(Línea 56)*

Define los códigos de error del sistema. BIEN (0) indica que no hubo error, y los demás se agrupan por centenas según la etapa en que se detectan:

- Archivos (100): el archivo no existe o no se indicó su nombre (NOMBRE_INCORRECTO, 100), no existe la estructura del archivo (NO_HAY_ARCHIVO, 101) o el archivo está vacío (NO_HAY_TEXTO, 102).
- Tokens (200): LINEA_VACIA (201) indica una línea vacía o con solo espacios.
- Léxico (300): un token no es instrucción, registro ni número (TIPO_PARAM_INVALIDO, 301), está en minúsculas (INICIA_MINUSCULA, 302), es un registro con mayúsculas y minúsculas mezcladas (REGISTRO_INVALIDO, 303) o es un número decimal (NUM_DECIMAL, 304).
- Sintáctico y ejecución (400): el primer parámetro no es un registro seguido de coma (INSTR_REG_INCUMPLIDA, 400); los parámetros son de un tipo incorrecto (PARAMETROS_INCORRECTOS, 401), insuficientes (PARAMETROS_INSUFICIENTES, 402) o de más (PARAMETROS_EXTRA, 403); se intentó dividir entre cero (DIV_ENTRE_CERO, 404); hay espacios de más o tabulaciones (ESPACIOS_EXTRA, 405); la línea no empieza con una instrucción válida (INSTRUCCION_NO_RECONOCIDA, 406); o hay espacios junto a la coma (COMA_CON_ESPACIOS, 407).
- Comandos (500): COMANDO_INVALIDO (500) indica que el comando escrito no es válido.

archivos.c

Archivo* crearArchivo() (Línea 22)

Descripción

Crea una estructura Archivo vacía, lista para recibir los renglones de un programa. Inicia los apuntadores al primer y al último renglón en NULL y el número de renglones en cero.

Parámetros: Ninguno.

Parámetros

Ninguno.

Valor de retorno

Apuntador a la nueva estructura Archivo, reservada con memoria dinámica. Se libera con liberarArchivo, que normalmente llama liberarEjecucion.

Llamada desde

main, al iniciar el programa y cada vez que el usuario usa el comando ejecutar.

Llama a

Ninguna función del proyecto. Usa la función de la biblioteca estándar malloc.

Renglon* crearRenglon(char *texto) (Línea 31)

Descripción

Crea e inicializa la estructura de un renglón del programa. Reserva memoria dinámica para una estructura Renglon, copia el texto recibido y elimina los caracteres de salto de línea \n y retorno de carro \r que puedan encontrarse al final del texto. Inicializa el estado de error en BIEN, el apuntador al siguiente renglón en NULL y los apuntadores al primer y último token en NULL.

Parametros

- texto (char *): texto correspondiente al renglón que se almacenará en la estructura.

Valor de retorno

Apuntador a la nueva estructura Renglon, reservada con memoria dinámica. Debe liberarse posteriormente mediante liberarRenglon.

Llamada desde

agregarRenglon, cada vez que se agrega un nuevo renglón al archivo.

Llama a

Ninguna función del proyecto. Usa las funciones de la biblioteca estándar malloc, strdup y strcspn.

Ejecucion* crearEjecucion(Archivo *prog, char *nombre) (Línea 3)

Descripción

Crea e inicializa la estructura de un programa en ejecución. Guarda el nombre del proceso sin la ruta (por ejemplo, de pruebas/a.asm guarda solo a.asm), asocia el programa recibido, coloca el IR en su primer renglón, inicia el PC, id y el contador de espera en cero y el estado en ESPERA. Además, reinicia en cero los registros AX, BX, CX y DX, por lo que el registro global reg debe haberse creado antes de llamarla.

Parámetros

- prog (Archivo *): programa que se va a ejecutar.
- nombre (char *): nombre o ruta del archivo del programa. Si es NULL, el nombre queda vacío.

Valor de retorno

Apuntador a la nueva estructura Ejecucion, reservada con memoria dinámica. Debe liberarse con liberarEjecucion.

Llamada desde

main, al iniciar el programa (con un nombre vacío) y cada vez que el usuario usa el comando ejecutar.

Llama a

Ninguna función del proyecto. Usa las funciones de la biblioteca estándar malloc, strchr y strdup.

void agregarRenglon(Archivo *archivo, char *texto) (Línea 44)

Descripción

Agrega un nuevo renglón al final de la lista enlazada de un archivo. Primero crea el renglón mediante crearRenglon, después lo enlaza al final de la lista. Si el archivo ya contiene renglones, conecta el nuevo elemento después del renglón actualmente señalado por final; si la lista está vacía, establece el nuevo renglón como inicio. Finalmente, actualiza “final” para que apunte al nuevo renglón e incrementa en uno el tamaño del archivo.

Parámetros

- archivo (Archivo *): archivo en cuya lista enlazada se agregará el nuevo renglón.
- texto (char *): texto correspondiente al renglón que se desea agregar.

Valor de retorno

No retorna ningún valor (void). La función modifica directamente la estructura Archivo, agregando el nuevo renglón y actualizando sus campos inicio, final y tamaño.

Llamada desde

leerArchivo, cada vez que se obtiene una nueva línea del archivo mediante getline. De esta manera, cada línea leída del archivo .asm se transforma en un elemento de la lista de renglones.

Llama a

crearRenglon, función del proyecto encargada de reservar e inicializar la estructura Renglon a partir del texto recibido. No utiliza directamente funciones adicionales de la biblioteca estándar dentro de esta función.

void liberarArchivo(Archivo *archivo) (*Línea 56*)

Descripción

Libera la memoria utilizada por un archivo. Primero recorre todos los renglones que contiene y los libera uno por uno. Al terminar, libera también la estructura Archivo.

Parámetros

- archivo (Archivo *): archivo que se va a liberar. Si es NULL, la función termina sin realizar ninguna operación.

Valor de retorno

Ninguno.

Llamada desde

liberarEjecucion, cuando se libera el programa asociado a una ejecución.

Llama a

liberarRenglon. Además, usa la función free de la biblioteca estándar.

void liberarEjecucion(Ejecucion *ejec) (Línea 68)

Descripción

Libera la memoria dinámica asociada a una estructura de ejecución. Primero verifica que el apuntador recibido no sea NULL; posteriormente libera el programa almacenado dentro de la ejecución mediante liberarArchivo, libera la memoria reservada para el nombre del proceso y, finalmente, libera la propia estructura Ejecucion. Esta función permite liberar de manera ordenada los recursos utilizados por una ejecución del simulador.

Parámetros

- ejec (Ejecucion *): apuntador a la estructura de ejecución que se desea liberar.

Valor de retorno

No retorna ningún valor (void). Su propósito es liberar los recursos asociados con la estructura Ejecucion.

Llamada desde

main, cuando se reemplaza la ejecución actual por una nueva al utilizar el comando ejecutar, y nuevamente al finalizar el programa para liberar los recursos restantes.

Llama a

liberarArchivo, función del proyecto que se encarga de liberar el Archivo asociado con la ejecución. También utiliza la función free de la biblioteca estándar de C para liberar la memoria correspondiente al nombre del proceso y a la estructura Ejecucion.

int leerArchivo(char *nomArchivo, Archivo *archivo); (Línea 74)

Descripción

Abre el archivo de programa indicado y lo lee línea por línea, agregando cada línea como un nuevo renglón al final de la estructura Archivo. Al terminar, cierra el archivo y libera la memoria usada para la lectura. Un archivo vacío no se considera error aquí; ese caso lo detecta después verifSintaxis.

Parámetros

- nomArchivo (char *): nombre o ruta del archivo que se va a leer.
- archivo (Archivo *): estructura donde se guardan los renglones leídos. Debe haberse creado antes con crearArchivo.

Valor de retorno

BIEN si el archivo se leyó correctamente; NO_HAY_ARCHIVO si la estructura archivo es nula; o NOMBRE_INCORRECTO si no se indicó un nombre o el archivo no se pudo abrir (por ejemplo, porque no existe).

Llamada desde

main, cuando el usuario usa el comando ejecutar.

Llama a

agregarRenglon. Además, usa las funciones de la biblioteca estándar fopen, getline, fclose y free.

tokens.c

int agregarTipoDato(Token *token) (*Línea 22*)

Descripción

Determina y asigna el tipo de dato correspondiente a un token durante el proceso de tokenización del programa. Para realizar la clasificación, llama a esInstruccion, que analiza el contenido textual del token y determina si corresponde a una instrucción, un registro, un registro acompañado de coma o un número. Cuando el resultado corresponde a uno de los tipos válidos (INSTR, REG, REG_COM o NUM), almacena dicho valor en el campo tipoParam de la estructura Token y devuelve BIEN. Si el tipo obtenido corresponde a un error léxico, devuelve directamente el código de error sin asignarlo al token.

Parámetros

- token (Token *): apuntador al token cuyo tipo de dato se desea determinar y almacenar.

Valor de retorno

Retorna BIEN cuando el token fue clasificado correctamente como instrucción, registro, registro con coma o número. Si la clasificación realizada por esInstruccion produce un error, retorna el código de error correspondiente.

Llamada desde

tokenizarRenglon, durante el proceso de separación de cada renglón en tokens. Después de crear un nuevo token y copiar su contenido, se llama a agregarTipoDato para clasificarlo antes de incorporarlo a la lista de tokens del renglón.

Llama a

esInstruccion, función del proyecto encargada de iniciar la clasificación léxica del token. No utiliza directamente funciones de la biblioteca estándar de C dentro de su implementación.

int tokenizar(Archivo *archivo) (*Línea 93*)

Descripción

Recorre todos los renglones de un archivo y realiza la separación de cada uno en tokens. El resultado del análisis de cada renglón se guarda en su campo error, permitiendo que después se pueda saber si el renglón es válido o contiene algún problema.

Parámetros

- archivo (Archivo *): archivo que contiene los renglones que se van a tokenizar.

Valor de retorno

BIEN cuando termina de procesar el archivo o NO_HAY_ARCHIVO si el apuntador recibido es NULL.

Llamada desde

main, después de leer el archivo indicado por el usuario y antes de verificar su sintaxis.

Llama a

tokenizarRenglon, una vez por cada renglón del archivo.

void agregarToken(Renglon *renglon, Token *token) (*Linea 12*)

Descripción

Agrega un token al final de la lista de tokens de un renglón. Si el renglón todavía no tiene tokens, el nuevo token se vuelve el primero; si ya tiene, se enlaza después del último. En ambos casos, el nuevo token queda registrado como el último del renglón, lo que permite agregar tokens sin recorrer toda la lista.

Parámetros

- renglon (Renglon *): renglón al que se agrega el token.
- token (Token *): token que se va a agregar. Debe haberse creado con crearToken, que deja su campo sig en NULL.

Valor de retorno

Ninguno.

Llamada desde

tokenizarRenglon, por cada token que el análisis léxico clasifica como válido.

Llama a
Ninguna función.

Token* crearToken() (Línea 3)

Descripción

Crea e inicializa la estructura de un token. Reserva memoria dinámica para una estructura Token, establece el texto del token en NULL, inicializa el tipo de parámetro en 0 y establece el apuntador al siguiente token en NULL.

Parámetros

Esta función no recibe parámetros.

Valor de retorno

Apuntador a la nueva estructura Token, reservada con memoria dinámica. Debe liberarse posteriormente como parte de la liberación del renglón mediante liberarRenglon.

Llamada desde

tokenizarRenglon, una vez por cada palabra del renglón. Después de crear el token, tokenizarRenglon le copia el texto, lo clasifica con agregarTipoDato y, si es válido, lo enlaza al renglón con agregarToken.

Llama a

Ninguna función del proyecto. Usa la función de la biblioteca estándar malloc.

void liberarRenglon(Renglon *renglon) (Línea 33)

Descripción

Libera la memoria utilizada por un renglón. Primero recorre la lista de tokens del renglón y libera el texto y la memoria de cada token. Después libera el texto original del renglón y finalmente la estructura Renglon.

Parámetros

- renglon (Renglon *): renglón que se va a liberar de memoria.

Valor de retorno

Ninguno.

Llamada desde

liberarArchivo, al recorrer y liberar todos los renglones de un archivo.

Llama a

Ninguna función del proyecto. Usa la función free de la biblioteca estándar.

static int tokenizarRenglon(Renglon *ren) (*Línea 14*)

Descripción

Separa un renglón en tokens y clasifica cada uno. Primero revisa si la línea está vacía o solo tiene espacios, y después comprueba el formato de espacios y coma con `verifEspacios`. Si la línea es correcta, hace una copia del texto en la que separa la coma del segundo parámetro (`AX,5` se convierte en `AX, 5`), para que el registro con coma y el valor queden como tokens distintos. Luego divide la copia en palabras; por cada una crea un token, lo clasifica con `agregarTipoDato` y, si es válido, lo agrega al renglón. Si un token no es válido, lo libera y detiene el proceso. Cuando el primer token no se reconoce, el error se reporta como operación no reconocida. Al terminar, libera la copia del texto.

Parámetros

- `ren (Renglon *)`: renglón que se va a separar en tokens.

Valor de retorno

`BIEN` si todos los tokens son válidos; `LINEA_VACIA` si la línea está vacía; `ESPACIOS_EXTRA` o `COMA_CON_ESPACIOS` si no respeta el formato de espacios; `INSTRUCCION_NO_RECONOCIDA` si el primer token no es válido; o el error léxico que regrese `agregarTipoDato` para los demás tokens.

Llamada desde

`tokenizar`, una vez por cada renglón del archivo.

Llama a

`verifEspacios`, `crearToken`, `agregarTipoDato` y `agregarToken`. Además, usa las funciones de la biblioteca estándar `strlen`, `strspn`, `malloc`, `strtok_r`, `strdup` y `free`.

lexicoSintactico.c

int esNumero(Token *token) (*Línea 14*)

Descripción

Determina si el texto de un token es un número entero válido. Acepta dígitos con un signo opcional al inicio (5, -3, +10). Si el texto tiene forma de número decimal (2.5, .5), lo reporta como error. Es el último paso de la cadena del análisis léxico: se llega aquí cuando el token no fue reconocido como instrucción ni como registro.

Parámetros

- token (Token *): token cuyo texto se va a revisar.

Valor de retorno

NUM si el texto es un número entero; NUM_DECIMAL si es un número decimal; o TIPO_PARAM_INVALIDO si no corresponde a ningún tipo válido (por ejemplo, 5a).

Llamada desde

esRegistro, cuando el token no es un registro.

Llama a

coincide, para comparar el texto con las expresiones regulares de número entero y de número decimal.

int esRegistro(Token *token) (*Línea 23*)

Descripción

Analiza el texto contenido en un token para determinar si corresponde a un registro válido, a un registro seguido de coma, a un registro escrito con mayúsculas y minúsculas incorrectas o a un número. Reconoce los registros AX, BX, CX y DX. Cuando el token contiene uno de estos registros seguido de una coma, elimina la coma del texto y lo clasifica como REG_COM. Si el registro utiliza una combinación de mayúsculas y minúsculas no permitida, lo clasifica como REGISTRO_INVALIDO. Si no corresponde a un registro, delega la clasificación a esNumero.

Parámetros

- token (Token *): apuntador al token cuyo texto se analizará para determinar su tipo.

Valor de retorno

Retorna REG si el token contiene un registro válido (AX, BX, CX o DX). Retorna REG_COM si contiene un registro seguido de una coma. Retorna REGISTRO_INVALIDO si el registro está escrito

con una combinación de mayúsculas y minúsculas no permitida. Si el texto no corresponde a un registro, retorna el resultado obtenido mediante `esNumero`.

Llamada desde

`esInstruccion`, cuando el token no corresponde directamente a una instrucción válida ni a una instrucción escrita en minúsculas.

Llama a

`esNumero`, para determinar si el token corresponde a un número cuando no coincide con ninguno de los formatos de registro. También utiliza las funciones de la biblioteca estándar `strchr`, `regcomp`, `regex` y `regfree` mediante la función auxiliar `coincide`.

int esInstruccion(Token *token) (Línea 39)

Descripción

Determina el tipo de elemento representado por un token dentro del análisis léxico del simulador. Primero verifica si el contenido del token corresponde a una de las instrucciones válidas del lenguaje (MOV, ADD, SUB, MUL, DIV, INC o DEC). Si coincide, retorna `INSTR`. En caso contrario, verifica si el token está formado únicamente por letras minúsculas y, si es así, retorna `INICIA_MINUSCULA`. Cuando ninguna de estas condiciones se cumple, delega la clasificación del token a la función `esRegistro`, que continúa el análisis determinando si corresponde a un registro, un registro acompañado de coma o un número.

Parámetros

- `token (Token *)`: apuntador al token cuyo contenido se desea analizar y clasificar.

Valor de retorno

Retorna `INSTR` cuando el token corresponde a una instrucción válida del lenguaje. Retorna `INICIA_MINUSCULA` cuando el contenido está escrito completamente en letras minúsculas. En cualquier otro caso, devuelve el resultado de `esRegistro`, que puede corresponder a un registro válido, un registro con coma, un número o alguno de los errores definidos para el análisis léxico.

Llamada desde

`agregarTipoDato`, durante la clasificación de cada token generado por `tokenizarRenglon`. Esta función constituye el punto inicial para determinar el tipo léxico de cada elemento del renglón.

Llama a

`esRegistro`, función del proyecto que continúa el análisis cuando el token no corresponde a una instrucción ni al patrón de letras minúsculas. También utiliza la función auxiliar `coincide`, definida en el mismo archivo, para realizar las comprobaciones mediante expresiones regulares.

int verifSintaxis(Archivo *archivo) (Línea 89)

Descripción

Verifica la sintaxis de todos los renglones del archivo. Recorre la lista de renglones y, si un renglón todavía no tiene un error, revisa que la instrucción y sus parámetros tengan la forma correcta. El resultado de esta revisión se guarda en el campo error de cada renglón.

Parámetros

- archivo (Archivo *): archivo que contiene las instrucciones que se van a verificar.

Valor de retorno

BIEN si la revisión se realizó; NO_HAY_ARCHIVO si la estructura recibida es NULL; o NO_HAY_TEXTO si el archivo no contiene ningún renglón.

Llamada desde

main, después de tokenizar el archivo cargado.

Llama a

verifRenglon, para revisar individualmente cada renglón que no tenga un error anterior.

int parserDosParametros(Token *token) (Linea 50)

Descripción

Verifica la sintaxis de las instrucciones que reciben dos parámetros (MOV, ADD, SUB, MUL y DIV). Revisa que después de la instrucción haya un registro seguido de coma, después un segundo parámetro que sea un registro o un número entero, y que no haya parámetros de más. Por ejemplo, MOV AX,5 y ADD AX,BX son válidas.

Parámetros

- token (Token *): primer token del renglón, que corresponde a la instrucción. Los parámetros se revisan a partir de su campo sig.

Valor de retorno

BIEN si la sintaxis es correcta; PARAMETROS_INSUFICIENTES si falta alguno de los dos parámetros (MOV, MOV AX,); INSTR_REG_INCUMPLIDA si el primer parámetro no es un registro seguido de coma (MOV AX 5); PARAMETROS_EXTRA si hay más de dos parámetros; o PARAMETROS_INCORRECTOS si el segundo parámetro no es un registro ni un número.

Llamada desde

verifRenglón, cuando la instrucción del renglón es MOV, ADD, SUB, MUL o DIV.

Llama a

Ninguna función.

int parserUnParametro(Token *token) (*Línea 63*)

Descripción

Verifica la sintaxis de las instrucciones que requieren un solo parámetro. Comprueba que exista un token después de la instrucción y que no exista un tercer token. Finalmente, verifica que el parámetro sea de tipo REG. Si todas las condiciones se cumplen, considera correcta la sintaxis de la instrucción.

Parámetros

- token (Token *): apuntador al token correspondiente a la instrucción que se está verificando. Se utiliza para acceder a los tokens siguientes y validar la cantidad y el tipo del parámetro.

Valor de retorno

Retorna BIEN cuando existe exactamente un parámetro y este es un registro. Retorna PARAMETROS_INSUFICIENTES cuando no existe un parámetro, PARAMETROS_EXTRA cuando existen parámetros adicionales y PARAMETROS_INCORRECTOS cuando el parámetro no corresponde a un registro.

Llamada desde

verifRenglón, al verificar las instrucciones INC y DEC, que requieren un solo parámetro.

Llama a

Ninguna función del proyecto. Accede directamente a los campos sig y tipoParam de la estructura Token.

int verifEspacios(char *texto) (*Línea 100*)

Descripción

Revisa que una línea respete el formato exacto de espacios del lenguaje: INSTR REG, VALOR o INSTR REG. No se permiten tabulaciones, espacios al inicio ni al final de la línea, espacios dobles, ni espacios antes o después de la coma. La revisión se hace sobre el texto original de la línea, antes de separarla en tokens.

Parámetros

- texto (char *): texto original de la línea, sin el salto de línea. No debe estar vacío, porque las líneas vacías se detectan antes de llamar a esta función.

Valor de retorno

BIEN si el formato es correcto; ESPACIOS_EXTRA si hay tabulaciones, espacios al inicio o al final, o espacios dobles; o COMA_CON_ESPACIOS si hay un espacio antes o después de la coma (MOV AX, 5, MOV AX ,5).

Llamada desde

tokenizarRenglon, después de comprobar que la línea no está vacía.

Llama a

espaciosMultiples, para detectar espacios dobles. Además, usa las funciones de la biblioteca estándar strlen y strchr.

int espaciosMultiples(char *texto) (*Línea 117*)

Descripción

Verifica si una cadena contiene dos caracteres de espacio consecutivos. Esta validación permite detectar el uso de espacios múltiples dentro de una instrucción, condición que el simulador considera un error de sintaxis. La función recorre el texto desde el primer carácter hasta encontrar el terminador '\0' y, en cada posición, comprueba si el carácter actual y el siguiente son ambos espacios. Cuando encuentra esta condición, detiene el recorrido y retorna ESPACIOS_EXTRA.

Parámetros

- texto (char *): cadena de caracteres que contiene el texto del renglón que se desea verificar.

Valor de retorno

Retorna ESPACIOS_EXTRA cuando encuentra dos espacios consecutivos en la cadena. Si recorre todo el texto sin detectar esta condición, retorna BIEN, indicando que no existen espacios múltiples de acuerdo con la validación implementada.

Llamada desde

verifEspacios, como parte de la revisión del formato de espacios de cada línea, antes de separarla en tokens.

Llama a

No llama a ninguna función del proyecto ni de la biblioteca estándar de C. Realiza directamente el recorrido de la cadena y la comparación de sus caracteres.

static int coincide(const char *texto, const char *patron) (Linea 14)

Descripción

Función auxiliar que indica si un texto coincide con una expresión regular. Compila el patrón recibido, compara el texto con él y libera la expresión compilada. Permite que las funciones del análisis léxico revisen los tokens con una sola llamada, sin repetir el proceso de compilar y liberar cada expresión.

Parámetros

- texto (const char *): texto que se va a revisar.
- patron (const char *): expresión regular extendida contra la que se compara el texto.

Valor de retorno

1 si el texto coincide con el patrón; 0 si no coincide o si el patrón no se pudo compilar.

Llamada desde

esNumero, esRegistro y esInstruccion.

Llama a

Ninguna función del proyecto. Usa las funciones regcomp, regex y regfree de la biblioteca regex.h.

static int verifRenglon(Renglon *ren) (Linea 73)

Descripción

Verifica la sintaxis de un solo renglón. Revisa que el renglón tenga tokens y que el primero sea una instrucción válida. Después, según la instrucción, revisa sus parámetros: MOV, ADD, SUB, MUL y DIV con parserDosParametros, e INC y DEC con parserUnParametro.

Parámetros

- ren (Renglon *): renglón cuya sintaxis se va a verificar.

Valor de retorno

BIEN si la sintaxis es correcta; LINEA_VACIA si el renglón no tiene tokens; INSTRUCCION_NO_RECONOCIDA si el primer token no es una instrucción; o el error que regrese parserDosParametros o parserUnParametro.

Llamada desde

verifSintaxis, por cada renglón que no tenga un error detectado durante la tokenización.

Llama a

parserDosParametros y parserUnParametro. Además, usa la función de la biblioteca estándar strcmp para identificar la instrucción.

instrucciones.c

Registros* crearRegistro() (Línea 3)

Descripción

Crea la estructura que contiene los registros utilizados por el simulador. Reserva memoria para Registros e inicializa EAX, EBX, ECX y EDX con un valor de cero.

Parámetros

No recibe parámetros.

Valor de retorno

Apuntador a la nueva estructura Registros, reservada con memoria dinámica.

Llamada desde

main, al iniciar el programa para crear la variable global de registros.

Llama a

Ninguna función del proyecto. Usa malloc para reservar memoria.

int ejecutarPrograma(Ejecucion *ejec) (Linea 13)

Descripción

Ejecuta la instrucción del renglón al que apunta el IR. Primero incrementa el PC, porque cada renglón cuenta como una instrucción aunque tenga error. Si el renglón trae un error detectado al cargar el archivo, no lo ejecuta y solo regresa ese error. Si es válido, identifica la instrucción por el texto de su primer token y llama a la función que la ejecuta. No avanza el IR al siguiente renglón; eso lo hace main después de mostrar el resultado.

Parámetros

- ejec (Ejecucion *): programa en ejecución. Su IR debe apuntar a un renglón válido (distinto de NULL).

Valor de retorno

BIEN si la instrucción se ejecutó correctamente. Si el renglón tenía un error de carga, regresa ese mismo código (por ejemplo, LINEA_VACIA o COMA_CON_ESPACIOS). Durante la ejecución puede regresar DIV_ENTRE_CERO (desde divR) o REGISTRO_INVALIDO (desde inc o dec). Si la instrucción no se reconoce, regresa INSTRUCCION_NO_RECONOCIDA.

Llamada desde

main, una vez por cada instrucción, cuando el contador de espera llega a 125 (aproximadamente cada dos segundos).

Llama a

mov, add, sub, mul, divR, inc y dec. Además, usa la función de la biblioteca estándar strcmp para identificar la instrucción.

int obtenerRegistro(char nombre[], Registros *registros, int **registro) (Línea 58)

Descripción

Obtiene la dirección de memoria del registro correspondiente al nombre recibido. Compara el nombre con AX, BX, CX y DX y, cuando encuentra una coincidencia, guarda en el parámetro registro la dirección del campo correspondiente dentro de la estructura de registros. Si el nombre no coincide con ninguno, no modifica ese parámetro.

Parámetros

- nombre (char []): nombre del registro que se desea localizar (AX, BX, CX o DX).
- registros (Registros *): estructura de registros donde se busca; en el proyecto se usa la variable global reg.
- registro (int **): apuntador donde se guarda la dirección del registro encontrado.

Valor de retorno

1 si el nombre corresponde a un registro válido; 0 si no corresponde a ninguno.

Llamada desde

obtenerValor, para leer el valor de un registro usado como segundo parámetro, y mov, add, sub, mul, divR, inc y dec, para obtener la dirección del registro destino.

Llama a

Ninguna función del proyecto. Utiliza la función de la biblioteca estándar strcmp.

int obtenerValor(Token *tok) (*Línea 84*)

Descripción

Obtiene el valor asociado al operando de una instrucción a partir de la información almacenada en un Token. La función determina el tipo de parámetro mediante el campo tipoParam del token. Cuando el token corresponde a un registro (REG), obtiene la dirección del registro mediante obtenerRegistro y retorna el valor almacenado en dicho registro. Cuando el token corresponde a un número (NUM), convierte su representación textual a un valor entero mediante atoi y lo retorna. Si el token no corresponde a ninguno de los tipos contemplados, retorna 0. Esta función permite centralizar la obtención del valor de los operandos y evitar la repetición del mismo bloque de código en las instrucciones MOV, ADD, SUB, MUL y DIV.

Parámetros

- tok (Token *): apuntador al token que contiene el operando cuyo valor se desea obtener. El token debe tener correctamente establecido su campo tipoParam como REG o NUM.

Valor de retorno

Retorna el valor entero correspondiente al operando. Si el token es de tipo REG, retorna el contenido del registro asociado. Si es de tipo NUM, retorna el número obtenido mediante la conversión de su texto. En cualquier otro caso, retorna 0.

Llamada desde

mov, add, sub, mul y divR, para obtener el valor del segundo operando de cada instrucción sin repetir la lógica de diferenciación entre registros y números.

Llama a

obtenerRegistro, función del proyecto utilizada para obtener la dirección del registro cuando el token corresponde a un registro. También utiliza atoi de la biblioteca estándar de C para convertir a entero el contenido textual de los tokens de tipo NUM.

int mov(Token *param) (*Línea 101*)

Descripción

Ejecuta la instrucción MOV. Obtiene el registro destino indicado por el primer parámetro y guarda en él el valor del segundo parámetro. Este valor puede ser un número entero o el contenido de otro registro.

Parámetros

- param (Token *): primer token de la instrucción MOV. Los parámetros de la instrucción se obtienen a partir de los siguientes tokens.

Valor de retorno

BIEN después de realizar la asignación.

Llamada desde

ejecutarPrograma, cuando la instrucción del renglón actual es MOV.

Llama a

obtenerRegistro, para obtener el registro destino, y obtenerValor, para obtener el valor del segundo parámetro.

int add(Token *param) (*Linea 110*)

Descripción

Ejecuta la instrucción ADD: suma el valor del segundo parámetro al registro indicado en el primero y guarda el resultado en ese mismo registro. El segundo parámetro puede ser un registro o un número entero. Por ejemplo, ADD AX,5 suma 5 a AX, y ADD AX,BX suma a AX el valor de BX.

Parámetros

- param (Token *): primer token del renglón, que corresponde a la instrucción. El registro destino está en param->sig y el segundo parámetro en param->sig->sig.

Valor de retorno

Siempre regresa BIEN. No valida los parámetros, porque verifSintaxis ya comprobó que el renglón es correcto antes de ejecutarlo.

Llamada desde

ejecutarPrograma, cuando la instrucción del renglón es ADD.

Llama a

obtenerRegistro, para obtener la dirección del registro destino, y obtenerValor, para obtener el valor que se va a sumar.

int sub(Token *param) (*Linea 119*)

Descripción

Ejecuta la instrucción SUB: resta el valor del segundo parámetro al registro indicado en el primero y guarda el resultado en ese mismo registro. El segundo parámetro puede ser un registro o un número entero. Por ejemplo, SUB AX,5 resta 5 a AX, y SUB AX,BX resta a AX el valor de BX.

Parámetros

- param (Token *): primer token del renglón, que corresponde a la instrucción. El registro destino está en param->sig y el segundo parámetro en param->sig->sig.

Valor de retorno

Siempre regresa BIEN. No valida los parámetros, porque verifSintaxis ya comprobó que el renglón es correcto antes de ejecutarlo.

Llamada desde

ejecutarPrograma, cuando la instrucción del renglón es SUB.

Llama a

obtenerRegistro, para obtener la dirección del registro destino, y obtenerValor, para obtener el valor que se va a restar.

int mul(Token *param) (Linea 128)

Descripción

Ejecuta la instrucción MUL: multiplica el registro indicado en el primer parámetro por el valor del segundo y guarda el resultado en ese mismo registro. El segundo parámetro puede ser un registro o un número entero. Por ejemplo, MUL AX,2 multiplica AX por 2, y MUL AX,BX multiplica AX por el valor de BX.

Parámetros

- param (Token *): primer token del renglón, que corresponde a la instrucción. El registro destino está en param->sig y el segundo parámetro en param->sig->sig.

Valor de retorno

Siempre regresa BIEN. No valida los parámetros, porque verifSintaxis ya comprobó que el renglón es correcto antes de ejecutarlo.

Llamada desde

ejecutarPrograma, cuando la instrucción del renglón es MUL.

Llama a

obtenerRegistro, para obtener la dirección del registro destino, y obtenerValor, para obtener el valor por el que se va a multiplicar.

int divR(Token *param) (Línea 137)

Descripción

Ejecuta la instrucción DIV. Obtiene el registro destino y el valor del segundo parámetro. Antes de realizar la división comprueba que el divisor no sea cero. Si es válido, divide el contenido del registro y guarda el resultado en el mismo registro.

Parámetros

- param (Token *): primer token de la instrucción DIV. Los parámetros de la operación se obtienen a partir de los siguientes tokens.

Valor de retorno

BIEN si la división se realizó correctamente o DIV_ENTRE_CERO si el valor utilizado como divisor es cero.

Llamada desde

ejecutarPrograma, cuando la instrucción del renglón actual es DIV.

Llama a

obtenerRegistro, para obtener el registro destino, y obtenerValor, para obtener el divisor.

int inc(Token *param) (*Linea 152*)

Descripción

Ejecuta la instrucción INC, incrementando en una unidad el contenido del registro especificado como parámetro. La función recibe como argumento el primer token de la instrucción y utiliza el siguiente token para identificar el registro que será modificado. Para localizar el registro utiliza obtenerRegistro; si el registro no puede ser encontrado, retorna REGISTRO_INVALIDO. Cuando el registro es válido, incrementa su contenido en una unidad y retorna BIEN.

Parámetros

- param (Token *): apuntador al primer token de la instrucción INC. El token siguiente (param->sig) contiene el nombre del registro que debe incrementarse.

Valor de retorno

Retorna BIEN cuando el registro fue localizado correctamente y su contenido fue incrementado. Retorna REGISTRO_INVALIDO cuando obtenerRegistro no logra identificar el registro indicado.

Llamada desde

ejecutarPrograma, cuando la instrucción almacenada en el primer token del renglón corresponde a INC. El analizador sintáctico verifica previamente que INC tenga exactamente un parámetro de tipo registro mediante parserUnParametro.

Llama a

obtenerRegistro, función del proyecto encargada de obtener la dirección del registro correspondiente al nombre proporcionado. También utiliza directamente los operadores de asignación y suma de C para incrementar el valor almacenado en el registro.

int dec(Token *param) (*Línea 165*)

Descripción

Ejecuta la instrucción DEC. Obtiene el registro indicado como parámetro y disminuye su valor en uno. Si el registro indicado no es válido, la operación no se realiza.

Parámetros

param (Token *): primer token de la instrucción DEC. El registro que se va a modificar se obtiene del siguiente token.

Valor de retorno

BIEN si la operación se realizó correctamente o REGISTRO_INVALIDO si no se pudo obtener el registro indicado.

Llamada desde

ejecutarPrograma, cuando la instrucción del renglón actual es DEC.

Llama a

obtenerRegistro, para obtener el registro que se va a modificar.

ventanas.c

int kbhit(void) (*Línea 49*)

Descripción

Revisa si el usuario presionó una tecla, sin detener el programa mientras espera. Consulta la entrada estándar con un tiempo de espera de cero: si hay un carácter disponible, lo indica de inmediato, y si

no, regresa sin bloquear. Esto permite que el ciclo principal siga ejecutando instrucciones mientras el usuario escribe. La función no lee el carácter; solo avisa que hay uno disponible, y después main lo lee con wgetch.

Parámetros

Ninguno.

Valor de retorno

1 si hay una tecla disponible para leer; 0 si no la hay o si ocurrió un error al consultar la entrada.

Llamada desde

main, en cada vuelta del ciclo principal.

Llama a

Ninguna función del proyecto. Usa la función del sistema select y las macros FD_ZERO, FD_SET y FD_ISSET para consultar la entrada estándar.

WINDOW* crearVentana(int altura, int anchura, int posY, char *nombre) (*Línea 67*)

Descripción

Crea y configura una ventana de la interfaz gráfica del programa utilizando ncurses. Reserva la ventana con las dimensiones y posición recibidas, dibuja un borde alrededor de ella, coloca el nombre de la ventana en la parte superior y actualiza su contenido en pantalla.

Parámetros

- altura (int): altura que tendrá la ventana.
- anchura (int): anchura que tendrá la ventana.
- posY (int): posición vertical donde se colocará la ventana.
- nombre (char *): texto que se mostrará como nombre o título de la ventana.

Valor de retorno

Apuntador de tipo WINDOW* correspondiente a la ventana creada mediante ncurses.

Llamada desde

main, al inicializar las ventanas principales de la interfaz del sistema.

Llama a

Ninguna función del proyecto. Utiliza las funciones de la biblioteca ncurses newwin, box, mvwprintw y wrefresh.

void impVentanaComandos(WINDOW *ventana) (*Línea 76*)

Descripción

Prepara y actualiza la ventana destinada a la introducción de comandos del simulador. La función coloca el indicador de entrada >, limpia el contenido restante de la línea, redibuja el borde de la ventana y establece nuevamente el título Comandos. Finalmente, actualiza la ventana para que todos los cambios realizados sean mostrados inmediatamente en la interfaz del simulador.

Parámetros

- ventana (WINDOW *): apuntador a la ventana de ncurses en la que se mostrará y preparará el área destinada a la captura de comandos.

Valor de retorno

No retorna ningún valor (void). Su función es modificar directamente el contenido y la presentación de la ventana recibida.

Llamada desde

main, cuando el usuario comienza a introducir un nuevo comando. Se utiliza para preparar visualmente la ventana antes de iniciar la captura de caracteres mediante leerComando.

Llama a

Utiliza las funciones mvwprintw, wclrtoeol, box y wrefresh de la biblioteca ncurses para escribir el indicador de entrada, limpiar la línea, dibujar el marco de la ventana y actualizar su contenido en pantalla. No llama a ninguna función definida en otro archivo del proyecto.

void limpiarComando(WINDOW *ventana) (*Línea 85*)

Descripción

Limpia el área donde el usuario escribe los comandos y vuelve a preparar la ventana para recibir uno nuevo. Después de limpiar la línea, dibuja nuevamente el borde, el título Comandos y el símbolo >.

Parámetros

- ventana (WINDOW *): ventana de comandos que se va a limpiar y volver a dibujar.

Valor de retorno

Ninguno.

Llamada desde

main, al iniciar el programa y después de que el usuario termina de ingresar un comando.

Llama a

Ninguna función del proyecto. Usa funciones de ncurses como wmove, wclrtoeol, box, mvwprintw y wrefresh.

void impEncabezado(WINDOW *ventana) (Línea 105)

Descripción

Dibuja el encabezado de la tabla de datos con los títulos de las columnas: ID, Nombre, PC, AX, BX, CX, DX, IR y Status. Crea temporalmente una subventana de tres renglones dentro de la ventana de datos, le dibuja un marco, coloca los separadores entre columnas y escribe cada título centrado en su columna. El ancho de cada columna se calcula a partir del ancho de la ventana, por lo que el encabezado se adapta al tamaño de la terminal. Al terminar, elimina la subventana; lo dibujado permanece en la ventana de datos.

Parámetros

- ventana (WINDOW *): ventana de datos donde se dibuja el encabezado.

Valor de retorno

Ninguno.

Llamada desde

main, al iniciar el programa y cada vez que se limpia la ventana de datos con el comando ejecutar.

Llama a

bordeColumna, para calcular la posición de cada separador, e imprimirCelda, para escribir cada título centrado. Además, usa las funciones de ncurses getmaxyx, derwin, box, mvwaddch, wrefresh, delwin y touchwin.

void impInstruccVentana(WINDOW *ventana, Ejecucion *ejecucion, int resultado) (Línea 134)

Descripción

Agrega una fila a la tabla de datos de la ventana para mostrar el resultado de la ejecución de un renglón del programa. Obtiene las dimensiones de la ventana y calcula el ancho disponible para la tabla. Mantiene la posición de la siguiente fila mediante variables estáticas y reinicia dicha posición cuando cambia la ejecución. Muestra el identificador, nombre del programa, contador de programa (PC), valores actuales de los registros AX, BX, CX y DX, instrucción registrada en el IR y el estado obtenido a partir del resultado de la ejecución. Finalmente, incrementa la fila para la siguiente instrucción y actualiza la ventana.

Parámetros

- ventana (WINDOW *): apuntador a la ventana de ncurses donde se mostrará la información.
- ejecucion (Ejecucion *): apuntador a la estructura que contiene la información del programa en ejecución, incluyendo su nombre, PC e IR.
- resultado (int): código obtenido después de ejecutar el renglón, utilizado para determinar el mensaje que se mostrará en la columna Status.

Valor de retorno

No retorna ningún valor. Actualiza directamente el contenido de la ventana recibida.

Llamada desde

main, después de ejecutar un renglón del programa, para mostrar en la tabla los valores de los registros y el estado correspondiente a la ejecución.

Llama a

imprimirCelda, imprimirNumero y mensajeEstatus, funciones utilizadas para mostrar los datos de la fila y convertir el código de resultado en el mensaje correspondiente. También utiliza las funciones de ncurses getmaxyx y wrefresh.

void limpiarVentana(WINDOW *ventana, char *nomVentana)
(Línea 96)

Descripción

Limpia completamente el contenido actual de una ventana de ncurses y posteriormente reconstruye los elementos básicos de su interfaz. Primero elimina todo el contenido de la ventana, después dibuja nuevamente el borde y coloca el nombre indicado en la parte superior. Finalmente, actualiza la ventana para reflejar los cambios en pantalla. Esta función permite reutilizar una misma ventana para mostrar nueva información sin conservar el contenido de una operación anterior.

Parámetros

- ventana (WINDOW *): apuntador a la ventana de ncurses cuyo contenido será eliminado y reconstruido.
- nomVentana (char *): cadena que contiene el título que debe mostrarse nuevamente en la parte superior de la ventana.

Valor de retorno

Ninguno.

Llamada desde

main, principalmente para limpiar la ventana de errores antes de procesar un nuevo comando y para reiniciar la ventana de datos cuando se comienza a cargar y ejecutar un nuevo archivo.

Llama a

Utiliza werase, box, mvwprintw y wrefresh de la biblioteca ncurses para borrar el contenido de la ventana, reconstruir su marco, imprimir el título proporcionado y actualizar la representación visual. No llama a ninguna función definida en otro archivo del proyecto.

static int bordeColumna(int i, int anchoTabla) (*Línea 10*)

Descripción

Calcula la posición horizontal del separador izquierdo de una columna de la tabla de datos. El ancho de cada columna es proporcional a su peso, definido en el arreglo pesos, de modo que la tabla ocupa todo el ancho disponible y se adapta al tamaño de la terminal. La posición se calcula sumando los pesos de las columnas anteriores. Si i es igual al número de columnas, regresa la posición del borde derecho de la tabla.

Parámetros

- i (int): número de la columna, empezando en 0.
- anchoTabla (int): ancho total de la tabla en caracteres.

Valor de retorno

Posición horizontal del separador, relativa al inicio de la tabla.

Llamada desde

impEncabezado, para colocar los separadores entre columnas, e imprimirCelda, para calcular dónde empieza cada celda y cuánto mide.

Llama a

Ninguna función.

static void imprimirCelda(WINDOW *ventana, int fila, int desp, int anchoTabla, int col, const char *texto, int centrar) (*Línea 21*)

Descripción

Escribe un texto dentro de una celda de la tabla de datos. Calcula dónde empieza la celda y cuánto mide, la limpia con espacios y después escribe el texto, ya sea centrado o alineado a la izquierda con un espacio de margen. Si el texto no cabe en la celda, lo recorta para que no invada la columna siguiente. Si el texto es NULL, deja la celda vacía.

Parámetros

- ventana (WINDOW *): ventana donde se escribe.
- fila (int): renglón de la ventana donde se escribe.
- desp (int): columna de la ventana donde empieza la tabla (0 para la subventana del encabezado y 1 para la ventana de datos).
- anchoTabla (int): ancho total de la tabla.
- col (int): número de la columna donde se escribe.
- texto (const char *): texto que se va a escribir.
- centrar (int): 1 para centrar el texto o 0 para alinearlo a la izquierda.

Valor de retorno

Ninguno.

Llamada desde

impEncabezado, para los títulos centrados; impInstruccVentana, para los datos de cada fila; e imprimirNumero.

Llama a

bordeColumna. Además, usa la función de ncurses mvwprintw y la función de la biblioteca estándar strlen.

static void imprimirNumero(WINDOW *ventana, int fila, int desp, int anchoTabla, int col, int valor) (*Línea 42*)

Descripción

Escribe un número entero dentro de una celda de la tabla de datos. Convierte el número a texto y lo escribe alineado a la izquierda mediante imprimirCelda.

Parámetros

- ventana (WINDOW *): ventana donde se escribe.
- fila (int): renglón de la ventana donde se escribe.
- desp (int): columna de la ventana donde empieza la tabla.
- anchoTabla (int): ancho total de la tabla.
- col (int): número de la columna donde se escribe.
- valor (int): número que se va a mostrar.

Valor de retorno

Ninguno.

Llamada desde

impInstruccVentana, para mostrar el PC y los valores de los registros AX, BX, CX y DX.

Llama a

imprimirCelda. Además, usa la función de la biblioteca estándar sprintf para convertir el número a texto.

comandos.c

void borrarCaracter(WINDOW *ventana, int *pos) (*Línea 3*)

Descripción

Elimina el último carácter introducido por el usuario en la línea de comandos de la interfaz del simulador. Para ello, primero decrementa la posición actual de escritura almacenada en pos, posteriormente reemplaza visualmente el carácter anterior por un espacio en la ventana y coloca nuevamente el cursor en esa posición. Finalmente, actualiza la ventana para mostrar inmediatamente el cambio realizado. La función forma parte del mecanismo de edición de comandos y permite implementar el comportamiento de la tecla de retroceso (Backspace).

Parámetros

- ventana (WINDOW *): apuntador a la ventana de ncurses donde se está introduciendo el comando y en la cual se eliminará visualmente el último carácter.
- pos (int *): apuntador a la posición actual dentro de la cadena de caracteres que está siendo capturada. La función decrementa este valor para retroceder una posición.

Valor de retorno

No retorna ningún valor (void). Modifica directamente el valor de pos y el contenido visual de la ventana recibida.

Llamada desde

leerComando, cuando detecta que el carácter recibido corresponde a KEY_BACKSPACE o al valor 127, y únicamente cuando la posición actual es mayor que cero. Esto evita intentar eliminar un carácter cuando no existe texto introducido.

Llama a

Utiliza las funciones mvwaddch, wmove y wrefresh de la biblioteca ncurses para sustituir visualmente el último carácter por un espacio, colocar el cursor en la nueva posición y actualizar la ventana. No llama a ninguna función definida en otro archivo del proyecto.

int detectarComando(char cad[]) (*Línea 31*)

Descripción

Analiza el comando escrito por el usuario. Primero elimina los espacios o tabulaciones que se encuentren al inicio y al final de la cadena. Después revisa si el comando es salir o si cumple con el formato ejecutar seguido del nombre de un archivo.

Parámetros

cad (char []): cadena que contiene el comando ingresado por el usuario.

Valor de retorno

SALIR si el comando escrito es salir; EJECUTAR_ARCHIVO si cumple con el formato de ejecutar un archivo; o COMANDO_INVALIDO si el comando no es reconocido.

Llamada desde

main, cuando el usuario presiona Enter después de escribir un comando.

Llama a

Ninguna función del proyecto. Usa funciones como strlen, memmove, strcmp, regcomp, regex y regfree.

char *sacarNomArchivo(char cad[]) (*Linea 54*)

Descripción

Obtiene el nombre del archivo a partir del comando ejecutar <archivo>. Busca el primer espacio o tabulación después de la palabra ejecutar, salta todos los espacios que haya y regresa la posición donde empieza el nombre. No copia el texto: el apuntador que regresa señala a una parte de la misma cadena del comando.

Parámetros

- cad (char []): cadena con el comando completo que escribió el usuario, ya sin espacios al inicio ni al final (los quita detectarComando).

Valor de retorno

Apuntador al inicio del nombre del archivo dentro de cad, o NULL si la cadena no contiene ningún espacio ni tabulación. No debe liberarse con free, porque no se reservó memoria nueva.

Llamada desde

main, cuando detectarComando identifica el comando ejecutar.

Llama a

Ninguna función del proyecto. Usa la función de la biblioteca estándar strpbrk para localizar el primer espacio o tabulación.

void leerComando(WINDOW *ventana, int *pos, char cad[], int caracter) (*Línea 19*)

Descripción

Procesa un carácter introducido por el usuario dentro de la ventana de comandos. Primero coloca el cursor en la posición correspondiente al texto que se está capturando y actualiza la ventana. Si el carácter corresponde a una tecla de retroceso, verifica que exista al menos un carácter escrito y, en ese caso, elimina el carácter anterior. Si se trata de un carácter ASCII imprimible y aún no se ha alcanzado el límite de 48 caracteres, lo agrega al arreglo cad, lo muestra en la ventana y avanza la posición de escritura.

Parámetros

- ventana (WINDOW *): apuntador a la ventana de ncurses donde se está capturando el comando.
- pos (int *): apuntador a la posición actual dentro de la cadena del comando. Se modifica conforme se agregan o eliminan caracteres.
- cad (char[]): arreglo donde se almacenan los caracteres introducidos por el usuario.
- caracter (int): código del carácter que se desea procesar. Puede corresponder a una tecla de retroceso o a un carácter ASCII imprimible.

Valor de retorno

No retorna ningún valor. Modifica la cadena cad, la posición pos y el contenido de la ventana según el carácter recibido.

Llamada desde

main, cada vez que se recibe un carácter mientras el usuario introduce un comando en la ventana de comandos.

Llama a

borrarCaracter, cuando se recibe una tecla de retroceso, e imprimirCaracter, cuando se recibe un carácter ASCII imprimible. También utiliza las funciones de ncurses wmove y wrefresh.

void imprimirCaracter(WINDOW *ventana, int *pos, char cad[], int caracter) (*Línea 11*)

Descripción

Guarda el carácter ingresado por el usuario dentro de la cadena del comando y lo muestra en la ventana. Después incrementa la posición para que el siguiente carácter se escriba en el lugar correspondiente.

Parámetros

- ventana (WINDOW *): ventana donde se muestra el carácter.
- pos (int *): posición actual dentro de la cadena del comando.
- cad (char []): cadena donde se está guardando el comando.
- caracter (int): carácter ingresado por el usuario.

Valor de retorno

Ninguno.

Llamada desde

leerComando, cuando el usuario introduce un carácter válido.

Llama a

Ninguna función del proyecto. Usa las funciones mvwaddch y wrefresh de ncurses.

logErrores.c

void detectarError(WINDOW *ventana, int error) (*Línea 3*)

Descripción

Muestra en la ventana de errores el mensaje correspondiente a un código de error. Primero limpia la ventana, de modo que solo queda visible el error más reciente, y después escribe el número del error seguido de su descripción (por ejemplo, "Error 404: No se puede dividir entre cero"). Si el código es BIEN, solo limpia la ventana sin escribir nada. Si el código no está contemplado, muestra "Error desconocido".

Parámetros

- ventana (WINDOW *): ventana de errores donde se muestra el mensaje.
- error (int): código de error, con un valor de errorAdvertencia.

Valor de retorno

Ninguno.

Llamada desde

main, en tres situaciones: cuando falla la carga de un archivo (por ejemplo, porque no existe), cuando una instrucción regresa un error al ejecutarse y cuando el usuario escribe un comando inválido.

Llama a

limpiarVentana, para borrar el mensaje anterior. Además, usa las funciones de ncurses mvwprintw y wrefresh.

const char *mensajeEstatus(int codigo) (*Línea 81*)

Descripción

Obtiene el mensaje de texto correspondiente a un código de resultado de la ejecución. Analiza el código recibido y lo relaciona con un estado determinado, como OK, Línea vacía, Op. desconocida, Div. entre 0, Error lexico o Error sintaxis. Si el código no corresponde a ninguno de los casos contemplados, retorna el mensaje Error.

Parámetros

- codigo (int): código de resultado que se desea convertir en un mensaje de estado.

Valor de retorno

Apuntador constante a una cadena de caracteres que representa el estado correspondiente al código recibido.

Llamada desde

impInstruccVentana, para obtener el texto que se muestra en la columna Status de la tabla de ejecución.

Llama a

Ninguna función del proyecto ni de la biblioteca estándar. Utiliza una estructura switch para determinar el mensaje correspondiente al código recibido.

prototipos.h

Prototipos de funciones

Contiene las declaraciones de las funciones utilizadas por los diferentes módulos del simulador. Permite que los archivos .c conozcan los tipos de parámetros y valores de retorno de las funciones que utilizan, manteniendo separadas las declaraciones de sus implementaciones.

Los prototipos se organizan según la responsabilidad de cada módulo:

Archivos (línea 11-18)

creación, lectura y liberación de archivos, renglones y ejecuciones.

Tokens (línea 20-25)

Creación, clasificación y liberación de tokens.

Léxico y sintáctico (línea 27-35)

Validación de instrucciones, registros, números, parámetros y espacios.

Instrucciones (línea 37-48)

Creación de registros, ejecución de operaciones y obtención de valores.

Ventanas (línea 50-57)

Creación, actualización y limpieza de la interfaz ncurses.

Comandos (línea 59-64)

Lectura, edición e interpretación de los comandos introducidos por el usuario.

Log de errores (línea 66-68)

Presentación de mensajes y estados asociados a los códigos de error.

También incluye las bibliotecas necesarias y estructuras.h, de manera que los prototipos puedan utilizar directamente las estructuras y tipos definidos por el proyecto.